Apuntes – MVVM y Repository

Desarrollo de Aplicaciones I · UADE · Guía de Ejercicios: MVVM y Repository (v1.0)

Proyecto: `MvvmLab` · Felipe Forte

Organización del proyecto en carpetas

El código está organizado por **capas**, con la misma estructura que el repositorio de referencia *PokedexAcademy-MVVM-Clean*. Cada carpeta es un *paquete* de Kotlin (`com.example.mvvmlab.presentation.nota`, etc.). En cada rama aparecen solo las carpetas que ese ejercicio usa.

```
app/src/main/java/com/example/mvvmlab/
 ├── MainActivity.kt          # punto de entrada: aloja la UI
 ├── MvvmLabApp.kt            # Application propia (desde el Ej 11)
 ├── core/
 │    ├── model/              # tipos comunes a varias pantallas (EstadoCarga)
 │    ├── navigation/         # rutas de navegación (desde el Ej 12)
 │    ├── theme/              # tema de Compose (Color, Theme, Type)
 │    └── util/               # utilidades (Fechas)
 ├── data/                    # CÓMO se guardan los datos
 │    ├── local/              # Room: Entity, DAO, Database (Ej 14–15)
 │    ├── mapper/             # Entity ⇄ modelo de dominio
 │    └── repository/         # implementaciones de los contratos (memoria, archivos, Room)
 ├── domain/                  # QUÉ es la app: Kotlin puro, sin Android
 │    ├── model/              # Nota, Registro, ResultadoLectura
 │    ├── repository/         # contratos (interfaces) de los Repository
 │    └── validacion/         # reglas de negocio (ValidadorNota, ValidadorRegistro)
 ├── presentation/            # MVVM: una carpeta por pantalla (UiState + ViewModel + Pantalla)
 └── di/                      # quién crea las dependencias (ContenedorApp, Ej 11)

app/src/test/java/com/example/mvvmlab/
 ├── domain/validacion/       # tests de reglas
 ├── fake/                    # Repository falsos para tests
 ├── presentation/            # tests de ViewModels
 └── util/                    # MainDispatcherRule
```

Capa	Regla
`domain/`	No importa nada de Android, Compose, Room ni archivos. Define los datos (`model`), los contratos (`repository`) y las reglas (`validacion`).
`data/`	Implementa los contratos de `domain`. Es la única capa que conoce archivos, SQL o Room.
`presentation/`	ViewModels y pantallas. Solo habla con los contratos de `domain`, nunca con `data` directamente.
`di/`	Conecta todo: elige qué implementación de `data` recibe cada ViewModel.

¿Y los *use cases* del repositorio de referencia?

El repositorio de referencia tiene además `domain/usecase/`. Acá no se agregaron: la guía pide no crear capas "porque sí", y en estos ejercicios cada use case solo llamaría a un método del Repository. La teoría (diapositiva 4) aclara que la capa de dominio es opcional.

Ejercicio 1 — ¿Quién está haciendo demasiado?

Consigna: tomar una pantalla donde todo está mezclado, marcar dónde se dibuja la interfaz, dónde se validan los datos, dónde se modifica la lista y dónde se guarda, y ubicar cada responsabilidad en *View*, *ViewModel* o *Model/Repository*. Todavía **no** se cambia la implementación.

Conceptos base

MVVM = separar en 3 responsabilidades

- View (Compose) **dibuja** el estado y **avisa** lo que hace el usuario (clicks, texto). No decide nada.
- ViewModel **recibe** la acción del usuario, **decide** (valida, coordina) y **expone** el estado de pantalla.
- Model / Repository **obtiene y guarda** los datos (memoria, archivo, Room, API). No sabe nada de pantallas.

UI = f(estado): la pantalla es una función del estado.

¿Cómo decido dónde va cada cosa?

El criterio **no** es "¿declara una variable?" — hay variables en las tres capas. Las preguntas útiles son:

1. **¿Esto dibuja algo?** → View. *(Test: si cambio Compose por una web, ¿tengo que reescribir esta línea? Sí → View.)*
2. **¿Es una regla que valdría igual en cualquier interfaz?** → ViewModel.
3. **¿Este dato tiene que sobrevivir si cierro la app?** Sí → dato real → Repository. No, solo importa mientras uso la pantalla → estado de pantalla → ViewModel.

Ejemplo: en WhatsApp, el mensaje que estás escribiendo (borrador) es de la pantalla; los mensajes ya enviados se guardan.

El ViewModel decide QUÉ mostrar. La View decide CÓMO mostrarlo.

"Hay un error: tarea vacía" → ViewModel. "El error va en rojo, debajo del botón" → View.

Código analizado (pantalla “mal hecha a propósito”)

```
@Composable
fun PantallaTareas(context: Context, modifier: Modifier = Modifier) {
    var texto by remember { mutableStateOf("") }                    // (A)
    var tareas by remember { mutableStateOf(listOf<String>()) }     // (B)
    var error by remember { mutableStateOf<String?>(null) }         // (C)

    Column(modifier.padding(16.dp)) {                                // (D)
        Text("Mis tareas (${tareas.size})")
        OutlinedTextField(value = texto, onValueChange = { texto = it })

        Button(onClick = {
            if (texto.isBlank()) {                                   // (E)
                error = "La tarea no puede estar vacía"
            } else if (tareas.contains(texto)) {                     // (F)
                error = "Esa tarea ya existe"
            } else {
                tareas = tareas + texto                              // (G)
                File(context.filesDir, "tareas.txt")                 // (H)
                    .writeText(tareas.joinToString("\n"))
                texto = ""                                           // (I)
                error = null
            }
        }) { Text("Agregar") }

        error?.let { Text(it, color = Color.Red) }                   // (J)
        LazyColumn { items(tareas) { Text(it) } }                    // (K)
    }
}
```

Resultado: tabla de responsabilidades (entregable)

Letra	Código	Qué hace	Dónde va
(A)	`var texto`	Borrador que escribe el usuario; no sobrevive al cierre	ViewModel
(B)	`var tareas`	Lista real de tareas; tiene que sobrevivir al cierre	Repository
(C)	`var error`	Mensaje de error actual (dato del estado de pantalla)	ViewModel
(D)	`Column`, `Text`, `OutlinedTextField`	Dibuja la pantalla	View
(E)	`texto.isBlank()`	Regla: no se agregan tareas vacías	ViewModel
(F)	`tareas.contains(texto)`	Regla: no se agregan tareas repetidas	ViewModel
(G)	`tareas = tareas + texto`	Agrega la tarea a la lista (el dueño del dato la modifica)	Repository
(H)	`File(...).writeText(...)`	Guarda las tareas en un archivo	Repository
(I)	`texto = ""`	Limpia el campo solo si la tarea se agregó bien (decisión sobre un dato del ViewModel)	ViewModel
(J)	`error?.let { Text(it, Color.Red) }`	Muestra el error en rojo (el error es del VM; dibujarlo es de la View)	View
(K)	`LazyColumn`	Muestra la lista en pantalla	View

Errores frecuentes (los que cometí y corregí)

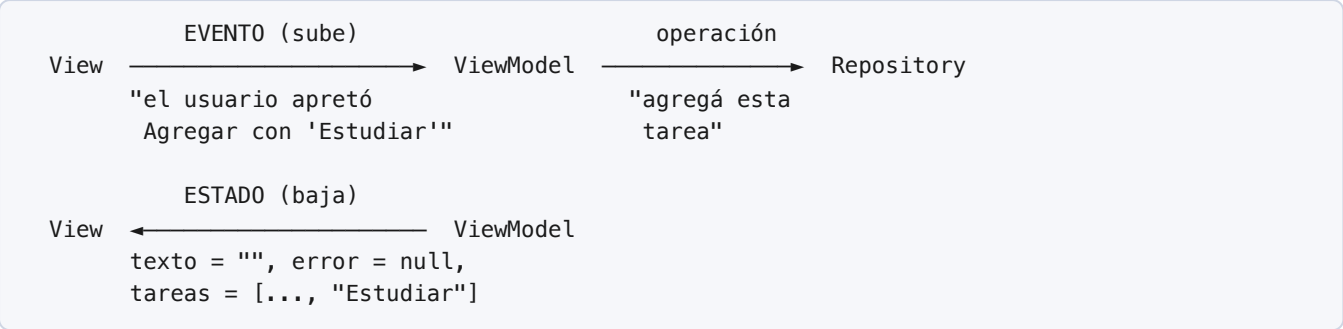
- Pensar que (A), (B) y (C) van al Repository "porque declaran variables". Solo (B) es un dato que debe persistir.
- Poner (I) en la View: `texto` es del ViewModel, y quien es dueño de un dato es quien lo modifica.
- Poner (J) en el ViewModel: el *error* es del ViewModel, pero *mostrarlo* con `Text` y `Color.Red` es de la View.

La cadena completa de "Agregar"

1. View el usuario aprieta *Agregar* → avisa al ViewModel.
2. ViewModel aplica las reglas (E) y (F). Si pasan → le pide al Repository que agregue; limpia el campo (I).
3. Repository agrega la tarea (G) y la guarda en el archivo (H).

Repository = **dueño del dato** · ViewModel = **intermediario** (lo pide y lo prepara) · View = **solo lo muestra**. La lista "verdadera" vive en un solo lugar: **fuente de verdad única**.

Desafío: las dos flechas (UDF – flujo unidireccional de datos)



> **Evento vs. estado**
>
> - **Evento** = algo que *pasó* (un click, una tecla). Es puntual.
> - **Estado** = cómo *está* la pantalla ahora. Se puede consultar en cualquier momento.
> - **UDF:** los eventos suben y el estado baja. La View nunca modifica el estado por su cuenta: solo avisa y redibuja.

Preguntas de la guía

1. ¿Qué trabajo tendría que seguir funcionando aunque mañana cambiemos Compose por otra interfaz?
Las **reglas** (E, F: no vacía, no repetida) y el **manejo de datos** (G, H: agregar y guardar). Es decir, todo lo que va en ViewModel y Repository.

2. ¿Qué datos existen sólo mientras se está escribiendo un formulario?
`texto` (el borrador) y `error` (el mensaje actual). Son estado de pantalla y no se guardan en el archivo.

> **Señal de alarma: `Context` en un composable**
>
> `PantallaTareas` recibe un `Context` solo para poder escribir el archivo (H). La guía advierte no aceptar soluciones que "llevan Context a todos los composables" o "abren archivos desde un botón". Es una pista de que el acceso a archivos está en el lugar equivocado: debería estar en el Repository.

Kotlin / Compose que apareció en el camino

Elemento	Qué es
`Unresolved reference: X`	Kotlin no sabe qué es X: falta el `import`, está mal escrito o la función/clase no existe.
`var x by remember { mutableStateOf(...) }`	Estado observable que sobrevive a la recomposición. El `by` necesita los imports `getValue` / `setValue`.
`LocalContext.current`	Forma de obtener el `Context` de Android dentro de un composable.
`modifier: Modifier = Modifier`	Convención: permite que quien llama le pase padding, tamaño, etc. (por ejemplo el `innerPadding` del Scaffold).
`@Preview` / `@Composable`	Anotaciones: "etiquetas" que siempre se aplican a una función. Solas, sin función debajo, no compilan.

Ejercicio 2 — Una pantalla no es su estado

Consigna: pantalla de notas con título, contenido y botón Guardar. Definir una `data class` de estado de pantalla con el contenido actual, el error de validación y si hay una operación en curso. Simular el guardado. Identificar eventos separados para editar el título, editar el texto y pulsar Guardar. Todavía **sin ViewModel**.
Archivo: `domain/model/Nota.kt + presentation/nota/` · Rama: `ej02-estado-pantalla`

> **Estado de pantalla (UiState)**
> Una sola "foto" de todo lo que la pantalla necesita para dibujarse. En el Ej 1 había 3 variables sueltas; ahora todo está junto en un objeto. **Si sabés el estado, sabés exactamente qué se ve.**

El estado

```
data class Nota(val titulo: String, val contenido: String)   // una nota YA guardada

data class NotaUiState(
    val titulo: String = "",          // lo escrito en el campo título
    val contenido: String = "",       // lo escrito en el campo contenido
    val error: String? = null,        // error de validación (null = sin error)
    val guardando: Boolean = false,   // operación en curso
    val guardada: Nota? = null        // última nota guardada (null = ninguna)
) {
    val hayCambiosSinGuardar: Boolean get() = ...   // propiedad CALCULADA
}
```

Elemento Kotlin	Por qué se usa
`data class`	Clase pensada para representar datos: genera `equals`, `toString` y `copy`.
Campos `val` + `copy(...)`	El estado es inmutable: para cambiarlo se crea una copia nueva con los campos modificados. Cada cambio es un estado nuevo y Compose detecta que tiene que redibujar.
`String?` / `Nota?`	Tipo anulable: el error o la nota guardada pueden no existir (`null`).
Valores por defecto (`= ""`)	`NotaUiState()` ya es el estado inicial: pantalla vacía.
Propiedad calculada (`get() = ...`)	Se deduce de otros campos en vez de guardarse: no hay información duplicada que pueda quedar desincronizada.

Desafío: tabla evento → cambio de estado (antes de codificar)

Evento	Cambio de estado esperado
Edita el **título**	`titulo = nuevo`, `error = null` (el error deja de tener sentido al escribir)
Edita el **contenido**	`contenido = nuevo`
Pulsa **Guardar** con título vacío	`error = "El título no puede estar vacío"`
Pulsa **Guardar** con título válido	`guardando = true`, `error = null`
Pulsa **Guardar** mientras ya guarda	Sin cambios: se ignora (y el botón está deshabilitado)
Termina el guardado simulado	`guardando = false`, `guardada = Nota(titulo, contenido)`

Cómo quedó organizado el código

Pieza	Rol
`Nota`	Modelo: una nota ya guardada.
`NotaUiState`	Estado de pantalla: **qué** se ve.
`alEditarTitulo`, `alEditarContenido`, `alPulsarGuardar`, `alTerminarGuardado`	Funciones Kotlin **puras**: reciben el estado actual y devuelven el nuevo. No conocen Compose. En el Ej 3 se mudan al ViewModel casi sin cambios.
`PantallaNotaConEstado`	Dueño del estado (provisorio, hasta el Ej 3) y del guardado simulado con `delay`.
`PantallaNota`	View **sin estado**: dibuja el `NotaUiState` que recibe y avisa los eventos por callbacks.
4 `@Preview`	Vacía · Error · Guardando · Guardada con cambios sin guardar.

State hoisting (elevar el estado)

```
@Composable
fun PantallaNota(
    estado: NotaUiState,                  // el estado BAJA
    onTituloChange: (String) -> Unit,     // los eventos SUBEN
    onContenidoChange: (String) -> Unit,
    onGuardar: () -> Unit,
    modifier: Modifier = Modifier
)
```

La View no guarda ni decide nada: recibe un valor y funciones (callbacks) para avisar. **Ventaja concreta:** como no tiene estado propio, se puede previsualizar en *cualquier* situación pasándole un `NotaUiState` armado a mano. Es el mismo UDF del Ej 1, ahora en código.

Lambdas y tipos función

`(String) -> Unit` = "una función que recibe un String y no devuelve nada". Se pasa como parámetro: `onTituloChange = { estado = alEditarTitulo(estado, it) }`. Dentro de la lambda, `it` es el único parámetro (el texto nuevo).

Guardado simulado: corrutina

`rememberCoroutineScope()` + `scope.launch { delay(1500); ... }` espera 1,5 s *sin congelar la pantalla*. Mientras tanto `guardando = true` → el botón muestra "Guardando..." y queda deshabilitado (`enabled = !estado.guardando`), lo que evita la doble pulsación. En el Ej 3 esto pasa a `viewModelScope`.

Preguntas de la guía

1. ¿El texto ingresado es lo mismo que la nota ya guardada?
No. `titulo` / `contenido` son lo que el usuario **está escribiendo** (cambia en cada tecla); `guardada` es la **última versión confirmada**. Por eso son campos distintos, y comparar ambos es lo que permite mostrar "Hay cambios sin guardar".

2. ¿Conviene mostrar el mensaje de error como parte del estado?
Sí. Si el error está en el estado, la pantalla se reconstruye completa a partir de `NotaUiState` (UI = f(estado)): sobrevive a la recomposición, se puede previsualizar y, con ViewModel (Ej 3), también a la rotación. Un mensaje "suelto" disparado desde el botón (un Toast) no se puede consultar ni recuperar.

Criterio de finalización

✅ Eventos reconocibles: `onTituloChange`, `onContenidoChange`, `onGuardar`.
✅ La pantalla se representa a partir de un único estado (`NotaUiState`): lo prueban las 4 previews, que dibujan cualquier situación sin ejecutar la app.

Ejercicio 2 — Explicación paso a paso

Recorrido del código del Ej 2, capa por capa: `domain/model/Nota.kt` (dato), `presentation/nota/NotaUiState.kt` (estado), `presentation/nota/NotaEventos.kt` (eventos) y `presentation/nota/PantallaNota.kt` (View).

Paso 0 · ¿Qué problema resuelve?

En el Ej 1 la pantalla tenía variables sueltas (`texto`, `tareas`, `error`): para saber qué se estaba viendo había que mirar tres cosas por separado, y cualquier parte del código podía cambiar cualquiera de ellas. La idea del Ej 2 es juntar todo en **un solo objeto**, el *estado de pantalla*. Si tenés ese objeto, sabés exactamente cómo se ve la pantalla: **UI = f(estado)**.

Paso 1 · `Nota`: el dato

```
data class Nota(
    val titulo: String,
    val contenido: String
)
```

Representa una **nota ya guardada**: el dato real, el que en ejercicios futuros va a ir al archivo. `data class` = clase cuyo propósito es guardar datos; Kotlin le genera `equals` (comparar), `toString` (imprimir) y `copy` (copiar cambiando algo). `val` = no se puede reasignar.

Paso 2 · `NotaUiState`: la "foto" de la pantalla

Campo	Pregunta que responde	Qué dibuja
`titulo`	¿Qué hay escrito en el campo título?	El texto dentro del campo
`contenido`	¿Qué hay escrito en el campo contenido?	El texto dentro del campo
`error`	¿Hay un error de validación?	El mensaje rojo debajo del título
`guardando`	¿Se está guardando ahora?	Botón "Guardando..." y deshabilitado
`guardada`	¿Cuál es la última nota guardada?	La tarjeta de abajo

- **`String?` y `Nota?`**: el `?` significa que puede ser `null` (puede no haber error, o ninguna nota guardada todavía).
- **Valores por defecto** (`= ""`, `= null`, `= false`): por eso `NotaUiState()` sin parámetros es el estado inicial, la pantalla vacía.
- **Todo es `val`**: el estado no se modifica, se **reemplaza por uno nuevo** con `copy`.

```
val antes = NotaUiState(titulo = "Hola")
val despues = antes.copy(error = "algo")   // mismo título, pero con error
// "antes" no cambió. "despues" es un objeto NUEVO.
```

¿Por qué así? Compose redibuja cuando el estado es **reemplazado por otro distinto**. Si se modificara un objeto por dentro, Compose podría no enterarse.

> **La propiedad calculada `hayCambiosSinGuardar`**
> No está entre los paréntesis: no se guarda, **se calcula** cada vez que alguien la lee, comparando `titulo` / `contenido` con `guardada`. Si fuera un campo más habría que acordarse de actualizarlo cada vez que cambia el título, el contenido o la nota guardada; un olvido y la pantalla muestra algo falso. Calculada, **no puede quedar desincronizada nunca**.

Paso 3 · Los eventos: "qué pasa cuando..."

Son el corazón del ejercicio. Todas tienen la misma forma: **reciben el estado actual → devuelven el estado nuevo**.

Función	Qué hace
`alEditarTitulo(estado, nuevo) = estado.copy(titulo = nuevo, error = null)`	Actualiza el título y **borra el error**: si decía "El título no puede estar vacío" y el usuario empieza a escribir, ese mensaje ya no tiene sentido.
`alEditarContenido(estado, nuevo) = estado.copy(contenido = nuevo)`	Solo cambia el contenido.
`alPulsarGuardar(estado)`	`when` evalúa en orden y usa la primera condición que se cumpla: 1) ¿ya se está guardando? → no hace nada; 2) ¿título vacío o solo espacios? → pone el error; 3) si no → `guardando = true`.
`alTerminarGuardado(estado)`	Deja de estar "guardando" y lo escrito pasa a ser la `guardada`.

> **Lo clave**
> Ninguna tiene `@Composable`, ni `Text`, ni nada de pantalla: **son Kotlin puro**. Son las reglas que en el Ej 1 dijimos que van al **ViewModel**. En el Ej 3 se mudan ahí casi sin cambios.

Paso 4 · `PantallaNota`: la View

```
@Composable
fun PantallaNota(
    estado: NotaUiState,                  // lo que tiene que dibujar  → el estado BAJA
    onTituloChange: (String) -> Unit,     // funciones para AVISAR     → los eventos SUBEN
    onContenidoChange: (String) -> Unit,
    onGuardar: () -> Unit,
    modifier: Modifier = Modifier
)
```

> **¿Qué es `(String) -> Unit`?**
> El **tipo** de una función: "recibe un String y no devuelve nada" (`Unit` ≈ `void` de Java). `() -> Unit` no recibe nada y no devuelve nada. A `PantallaNota` se le pasan **funciones como parámetros**, y ella las llama cuando el usuario hace algo.

Adentro, cada pieza lee del estado:

```
OutlinedTextField(
    value = estado.titulo,                                  // muestra lo que dice el estado
    onValueChange = onTituloChange,                         // cuando el usuario escribe, AVISA
    isError = estado.error != null,                         // si hay error, borde rojo
    supportingText = { estado.error?.let { Text(it) } }  // y el mensaje
)
Button(
    onClick = onGuardar,                                    // cuando aprieta, AVISA
    enabled = !estado.guardando                             // deshabilitado mientras guarda
) { Text(if (estado.guardando) "Guardando..." else "Guardar") }
```

Lo que NO hace: no valida, no guarda, no decide nada. Si el usuario escribe, no cambia el título: solo avisa "escribieron esto". Esto se llama **state hoisting** (elevar el estado): la pantalla no es dueña de su estado, lo recibe desde arriba.

Paso 5 · `PantallaNotaConEstado`: el dueño (provisorio)

```
@Composable
fun PantallaNotaConEstado(modifier: Modifier = Modifier) {
    var estado by remember { mutableStateOf(NotaUiState()) }
    val scope = rememberCoroutineScope()

    PantallaNota(
        estado = estado,
        onTituloChange = { estado = alEditarTitulo(estado, it) },
        onContenidoChange = { estado = alEditarContenido(estado, it) },
        onGuardar = { ... },
    )
}
```

- `mutableStateOf`: Compose **observa** esta variable; cuando cambia, redibuja.
- `remember`: hace que el valor **no se pierda** cuando Compose redibuja.
- `{ estado = alEditarTitulo(estado, it) }` es una **lambda** (función sin nombre); `it` es el parámetro que recibe (el texto nuevo). Se lee: "cuando cambie el título, reemplazá el estado por el resultado de `alEditarTitulo`".
- En el Ej 3 este rol (dueño del estado) lo toma el **ViewModel**.

Paso 6 · El guardado simulado

```
onGuardar = {
    estado = alPulsarGuardar(estado)            // 1. aplicar la regla
    if (estado.guardando) {                     // 2. ¿pasó la validación?
        scope.launch {                          // 3. arrancar tarea en segundo plano
            delay(1500)                         // 4. esperar 1,5 s (simula el disco)
            estado = alTerminarGuardado(estado) // 5. listo
        }
    }
}
```

¿Por qué `scope.launch`? Si se esperara 1,5 s "normalmente", **la pantalla se congelaría**. `launch` arranca una **corrutina**: una tarea que espera sin bloquear la pantalla. Por ahora alcanza con: *"launch + delay = esperar sin congelar la app"* (se profundiza en el Ej 9).

Paso 7 · Recorrido completo de un click

Título “Parcial” escrito, el usuario aprieta **Guardar**:

```
1. VIEW      Button → llama onGuardar()                              (EVENTO sube)
2. DUEÑO     estado = alPulsarGuardar(estado)
3. REGLA     título "Parcial" no está vacío → copy(guardando = true)
4. COMPOSE   detecta estado nuevo → redibuja PantallaNota
5. VIEW      lee estado.guardando = true → botón "Guardando..." deshabilitado
                                                                     (ESTADO baja)
   ... 1,5 segundos ...
6. DUEÑO     estado = alTerminarGuardado(estado)
7. REGLA     copy(guardando = false, guardada = Nota("Parcial", ""))
8. COMPOSE   redibuja
9. VIEW      botón vuelve a "Guardar" + aparece la tarjeta "Última nota guardada"
```

Nadie toca la pantalla directamente. Solo se cambia el estado, y la pantalla se redibuja sola a partir de él.

Paso 8 · Los Previews

```
@Preview(name = "2. Error de validación")
@Composable
fun PantallaNotaErrorPreview() {
    MvvmLabTheme {
        PantallaNota(
            NotaUiState(
                contenido = "Texto sin título",
                error = "El título no puede estar vacío"
            ),
            {}, {}, {}
        )
    }
}
```

Como `PantallaNota` solo dibuja lo que le pasan, se puede armar un estado a mano con cualquier situación y **verla sin correr la app**. Los `{}` son lambdas vacías: en el preview los botones no necesitan hacer nada. Es la **prueba del criterio de finalización**: si cualquier situación se dibuja pasando solo un `NotaUiState`, la pantalla se representa a partir de un único estado.

Resumen

1. **`NotaUiState`** = todo lo que la pantalla necesita, en un objeto. Inmutable: se cambia con `copy`.
2. **Funciones de evento** = reglas puras *estado → estado nuevo*. Sin Compose. En el Ej 3 van al ViewModel.
3. **`PantallaNota`** = View sin estado. Recibe `estado` (baja) y callbacks (suben). No decide nada.
4. **`PantallaNotaConEstado`** = dueño provisorio del estado. Conecta eventos con reglas.
5. **Nunca se toca la pantalla**: se cambia el estado y Compose redibuja.

Para comprobar que se entendió

Si en `alEditarTitulo` se sacara el `error = null` y quedara solo `estado.copy(titulo = nuevo)`, ¿qué pasaría en la pantalla al apretar Guardar con el título vacío y después empezar a escribir?